

PRESIDENCY UNIVERSITY

CSE322: Microprocessors Based System Design Laboratory

Class 01: Registers, Counters & Buses

Computer Organization Fundamentals for Microprocessors

Afzal Hossan

Lecturer

Department of Computer Science and Engineering

Presidency University

Class 01: Learning Objectives

By the end of today's laboratory, students should be able to:

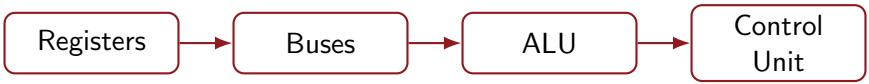
1. Explain the purpose of **registers** in a processor.
2. Describe **register transfer operations**.
3. Explain the role of a **bus** in transferring information.
4. Distinguish between:
   - Data bus
   - Address bus
   - Control bus
5. Explain the basic operation of a **counter**.
6. Describe the roles of the **ALU** and control unit.
7. Trace a simple **instruction execution cycle**.
8. Relate register and bus operations to the execution of **assembly instructions**.

From Digital Logic to a Processor

A processor is not a single component.

It is a collection of components that cooperate to:

Store $\rightarrow$ Move $\rightarrow$ Process $\rightarrow$ Control



Today's Focus

We will study the basic components that allow a processor to execute instructions.

What is a Register?

Definition

A **register** is a small, high-speed storage location inside the processor used to temporarily hold data, addresses, or control information.

A register can store:

- Data being processed
- Intermediate results
- Memory addresses
- Instructions
- Status information

Register = Fast temporary storage inside CPU

Register Size

The size of a register determines how many bits it can hold.

Register Size	Number of Bits	Example
4-bit	4	1010
8-bit	8	10110101
16-bit	16	1011010100110011
32-bit	32	1011...
64-bit	64	1011...

For an n-bit register:

$$\boxed{\text{Number of possible values} = 2^n}$$

For example, an 8-bit register can represent:

$$2^8 = 256$$

different binary patterns.

Types of Registers

Different registers serve different purposes.

Register	Purpose
General-purpose register	Holds data and intermediate results
Accumulator	Holds arithmetic/logic results
Program Counter (PC)	Holds address of next instruction
Instruction Register (IR)	Holds current instruction
Memory Address Register (MAR)	Holds memory address
Memory Data Register (MDR)	Holds data transferred to/from memory
Status/Flag Register	Holds processor status information

Important

The exact set of registers depends on the processor architecture. We will study the actual 8086 registers in a later class.

Register Transfer

Information can be transferred from one register to another.
Suppose we have two registers:

$$R_1 \qquad R_2$$

A transfer from R_2 to R_1 can be represented as:

$$\boxed{R_1 \leftarrow R_2}$$

This means:

The contents of R_2 are copied into R_1.

Example

Before transfer:

$R_1 = 0011\,1010$

$R_2 = 1100\,0101$

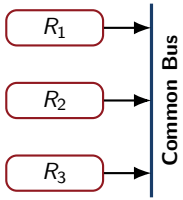
After ($R_1 \leftarrow R_2$):

$\boxed{R_1 = 1100\,0101}$

$R_2 = 1100\,0101$ (unchanged)

Register Transfer Through a Bus

Registers do not necessarily connect directly to every other register. A **common bus** provides a shared path for data transfer.



The bus allows one selected source to place information on a shared communication path.

Example: Register Transfer

Suppose:

$$R_1 = 0101\,1010$$

and

$$R_2 = 0011\,1100$$

We want to perform:

$$\boxed{R_1 \leftarrow R_2}$$

Conceptually:



After the transfer:

$$\boxed{R_1 = 0011\,1100}$$

What is a Bus?

Definition

A **bus** is a collection of parallel communication lines used to transfer information between components of a computer.

A typical computer system uses three major types:

Data Bus	Address Bus	Control Bus

Think of a Bus as a Highway

Data, addresses, and control information travel between different components through appropriate buses.

Data Bus

The **data bus** carries the actual information being transferred.

Examples:

- Data from memory to CPU
- Data from CPU to memory
- Data between registers and other components
- Data exchanged with I/O devices

If a processor has a:

16-bit data bus

then it can transfer up to 16 bits of data in one bus transfer.



Address Bus

The **address bus** identifies the location involved in a memory or I/O operation.
For a memory access, the processor needs to specify:

$$\boxed{\text{Where?}}$$

The answer is provided through the address bus.
If there are n address lines:

$$\boxed{\text{Maximum addressable locations} = 2^n}$$

For example:

$$\text{20-bit address bus} \Rightarrow 2^{20} = 1,048,576$$

addressable locations.

Control Bus

The **control bus** carries signals that control and coordinate operations.
Examples include:

- Read
- Write
- Clock
- Interrupt
- Reset
- Bus control signals

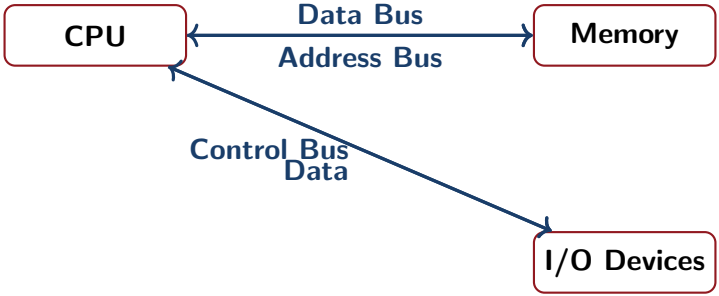
For example, during a memory read:

Address $\rightarrow$ Memory

Read Control Signal $\rightarrow$ Memory

Data $\leftarrow$ Memory

Data, Address & Control Buses



Bus	Main Purpose
Data	Carries information
Address	Identifies location
Control	Coordinates operations

What is a Counter?

Definition

A **counter** is a sequential digital circuit that progresses through a predetermined sequence of states, usually according to clock pulses.

A simple binary up-counter:

$$0000 \rightarrow 0001 \rightarrow 0010 \rightarrow 0011 \rightarrow 0100 \rightarrow \cdots$$

Each clock pulse causes the counter to advance.

$$\boxed{\text{Current State} \xrightarrow{\text{Clock}} \text{Next State}}$$

Program Counter

A processor uses a special counter called the:

Program Counter (PC)

The PC contains the address of the instruction that will be fetched next.

For example:

Step	PC	Instruction
1	1000H	MOV
2	1003H	ADD
3	1005H	SUB
4	1007H	HLT

After fetching an instruction, the processor normally updates the PC so that it points to the next instruction.

Arithmetic Logic Unit (ALU)

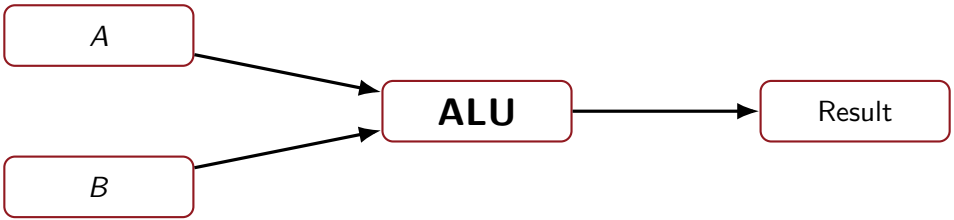
The **Arithmetic Logic Unit** performs arithmetic and logical operations.

Arithmetic

- Addition
- Subtraction
- Increment
- Decrement

Logical

- AND
- OR
- XOR
- NOT
- Comparison



Control Unit

The **control unit** coordinates the activities of the processor.

It determines:

- Which register should provide data
- Which register should receive data
- Which ALU operation should be performed
- When memory should be read or written
- Which control signals should be activated

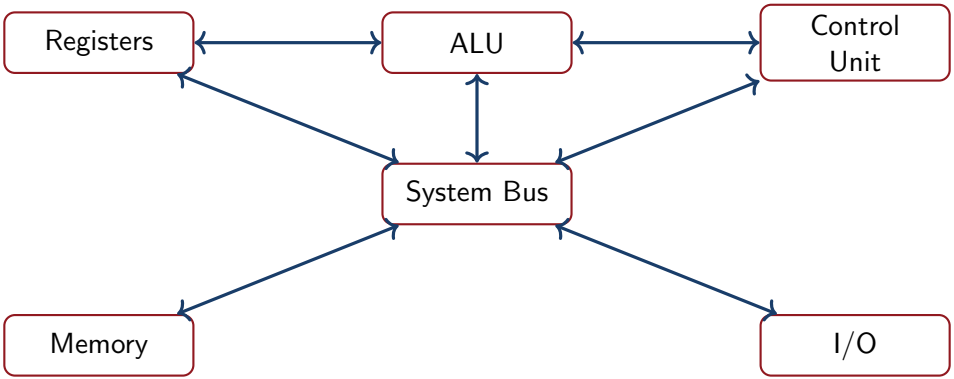
Instruction $\rightarrow$ Control Unit $\rightarrow$ Control Signals

Simple Interpretation

The ALU performs the operation; the control unit tells the processor **when and how**.

Basic CPU Organization

The basic components work together.



Registers + ALU + Control + Buses + Memory + I/O

form the basic foundation of a computer system.

Instruction Cycle

Every instruction goes through a sequence of basic stages.

$$\boxed{\text{FETCH}} \rightarrow \boxed{\text{DECODE}} \rightarrow \boxed{\text{EXECUTE}}$$

1. **Fetch**
   - Obtain the instruction from memory.
2. **Decode**
   - Determine what operation the instruction specifies.
3. **Execute**
   - Perform the required operation.

Today's Important Connection

Assembly instructions are eventually translated into sequences of hardware operations involving registers, buses, ALU and control signals.

Example: Fetching an Instruction

Suppose:

$$PC = 1000H$$

and memory contains:

$$M[1000H] = \text{Instruction}$$

The processor needs to fetch it.
Conceptually:

$$\boxed{PC \rightarrow MAR}$$

$$\boxed{MAR \rightarrow \text{Memory Address}}$$

$$\boxed{\text{Memory Data} \rightarrow IR}$$

Instruction Execution: A Simple Example

Consider the conceptual operation:

$$\boxed{R_3 \leftarrow R_1 + R_2}$$

The processor must:

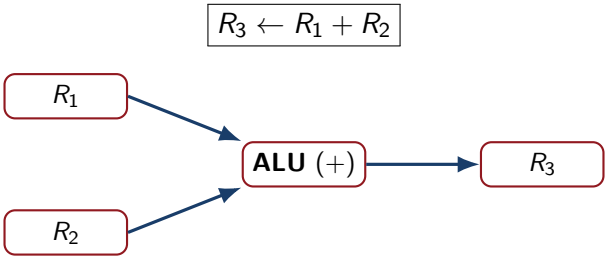
1. Read the contents of R_1.
2. Read the contents of R_2.
3. Provide both values to the ALU.
4. Perform addition.
5. Store the result in R_3.

Conceptually:

$$R_1, R_2 \rightarrow ALU \rightarrow R_3$$

Putting Everything Together

Consider:



- **Registers** hold the data.
- **Bus** provides the transfer path.
- **ALU** performs the addition.
- **Control Unit** coordinates the operation.

Laboratory Exercise 1: Register Transfer

Assume three 8-bit registers:

$$R_1, \quad R_2, \quad R_3$$

Initially:

$$R_1 = 10101100$$

$$R_2 = 01100101$$

$$R_3 = 00000000$$

Perform the following operations sequentially:

1. $R_3 \leftarrow R_1$
2. $R_1 \leftarrow R_2$
3. $R_2 \leftarrow R_3$

Task

Record the contents of all three registers after each operation.

Laboratory Exercise 2: Simple ALU Operation

Assume:

$$R_1 = 00001101$$

$$R_2 = 00000111$$

Perform:

$$\boxed{R_3 \leftarrow R_1 + R_2}$$

Determine:

1. What value should be placed on the bus from R_1?
2. What value should be placed on the bus from R_2?
3. What operation should the ALU perform?
4. What is the final value of R_3?

$$00001101_2 + 00000111_2 = \boxed{00010100_2}$$

Laboratory Exercise 3: Counter

Consider a 4-bit counter initialized to:

0000

Record its state after each clock pulse.

Clock Pulse	Counter State
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000

Laboratory Exercise 4: Instruction Trace

Consider the following conceptual instruction sequence:

Step	Operation
1	$R_1 \leftarrow 05H$
2	$R_2 \leftarrow 03H$
3	$R_3 \leftarrow R_1 + R_2$
4	$R_4 \leftarrow R_3$

For every step, record:

- Register values
- Source register
- Destination register
- ALU operation, if any
- Data movement through the bus

Common Mistakes to Avoid

Mistake 1

Confusing a **register** with memory.

A register is inside the processor and is much smaller and faster than main memory.

Mistake 2

Thinking that a bus stores data.

A bus is primarily a **communication path**.

Mistake 3

Confusing address and data.

Address = Where	Data = What

Connection to SAP-1

The concepts from today's class directly prepare us for SAP-1.

SAP-1 contains:

Component	Role
Accumulator	Stores intermediate data
B Register	Holds ALU operand
Program Counter	Tracks next instruction
Instruction Register	Holds current instruction
Memory Address Register	Holds memory address
Output Register	Holds output value
Bus	Transfers information
ALU	Performs arithmetic operations
Control Unit	Generates control signals

Key Takeaways

1. **Registers** provide fast temporary storage inside the processor.
2. **Register transfer** describes movement of data between registers.
3. A **bus** provides a communication path between components.
4. The **data bus** carries data.
5. The **address bus** identifies locations.
6. The **control bus** carries coordination signals.
7. A **counter** progresses through a sequence of states according to clock pulses.
8. The **ALU** performs arithmetic and logical operations.
9. The **control unit** coordinates processor operations.
10. Instructions are executed through a sequence of **fetch, decode and execute** operations.

Next Class: SAP-1 Architecture

Registers → Bus → ALU → Control

⇓

SAP-1

We will study how these basic components are combined to create a simple programmable computer.

Preparation

Review binary numbers, registers, buses, ALU operations, and the basic fetch–decode–execute cycle.